Let's work on a simple AI-assisted Python security program.

Scenario: Your organization has had several instances of modified files being used after downloading. These files were corrupted during downloading.

Your fellow employees don't understand the concept of hashing.

Task: Create a Hash Calculator that will hash data by using: MD5, SHA-128, SHA -256

Also, build a file integrity checker that employees can use to ensure files haven't been changed during download,

Task: Create a Build a Hash Calculator / File Integrity Checker.

Use the Prompt --> Generate-->Understand-->Test-->Verify-->Security Functions-->Critical Thinking model

**Create five test cases **

Let's build this step by step for your **CIS 226 Programming for Cybersecurity** class using Python in Spyder 6. Since you're learning Python, we'll keep the program beginner-friendly and include comments explaining what each part does.

One important detail: **SHA-128 is not a standard SHA hash algorithm name.** Your assignment may mean SHA-1, which produces a 160-bit hash, or another algorithm. We'll use MD5, SHA-1, and SHA-256 in this example, but you should confirm whether your instructor intended SHA-1 or a different algorithm.

1. Project: Hash Calculator and File Integrity Checker

Objective

Create a Python program that calculates file hashes and helps employees verify that downloaded files have not been modified or corrupted.

What the program will do

- Calculate MD5, SHA-1, and SHA-256 hashes.

- Allow employees to enter text and calculate its hashes.
- Calculate hashes for downloaded files.
- Compare a file's hash against a previously recorded hash.
- Display whether the file's hash matches the expected value.
- Explain why hashing is important in cybersecurity.

Security note: MD5 and SHA-1 are considered cryptographically weak. SHA-256 is the recommended choice of these three for file integrity verification. Hashing can detect changes when the expected hash comes from a trusted source, but it does not prove a file is safe or free of malware.

---

2. Python Code for Spyder 6

Open Spyder 6, create a new Python file, and save it as `hash_integrity_checker.py`.

Copy the following code into the editor.

Writing



python

```python
# Project: Hash Calculator and File Integrity Checker
# Course: CIS 226 - Programming for Cybersecurity

import hashlib
import os

# Display the program's welcome message
print("=" * 50)
print("     CYBERSECURITY HASH CALCULATOR")
print("        File Integrity Checker")
print("=" * 50)


# Function to calculate hashes for text
def calculate_text_hashes(text):
    # Convert the text into bytes
    data = text.encode("utf-8")

    # Calculate the hashes
```

```
    md5_hash = hashlib.md5(data).hexdigest()
    sha1_hash = hashlib.sha1(data).hexdigest()
    sha256_hash = hashlib.sha256(data).hexdigest()

    # Display the results
    print("\nHash Results:")
    print("MD5:    ", md5_hash)
    print("SHA-1:  ", sha1_hash)
    print("SHA-256:", sha256_hash)


# Function to calculate the SHA-256 hash of a file
def calculate_file_hash(filename):
    sha256 = hashlib.sha256()

    # Read the file in small pieces
    with open(filename, "rb") as file:
        while True:
            data = file.read(4096)

            # Stop when the end of the file is reached
            if not data:
                break

            sha256.update(data)

    return sha256.hexdigest()


# Function to check file integrity
def check_file_integrity():
    filename = input("\nEnter the full path to the file: ")

    # Check whether the file exists
    if not os.path.isfile(filename):
        print("Error: File not found.")
        return
```

```
    # Calculate the file's current SHA-256 hash
    actual_hash = calculate_file_hash(filename)

    print("\nCalculated SHA-256 hash:")
    print(actual_hash)

    # Ask for the original trusted hash
    expected_hash = input(
        "\nEnter the original SHA-256 hash: "
    ).strip()

    # Validate the expected hash format
    if len(expected_hash) != 64:
        print("Error: SHA-256 hashes must contain 64 characters.")
        return

    if any(c not in "0123456789abcdefABCDEF"
           for c in expected_hash):
        print("Error: The hash contains invalid characters.")
        return

    # Compare the hashes
    if actual_hash.lower() == expected_hash.lower():
        print("\nRESULT: Hashes match.")
        print("The file has not changed relative to the trusted
hash.")
    else:
        print("\nWARNING: Hashes do not match!")
        print("The file may have changed or become corrupted.")


# Main program menu
while True:
    print("\nChoose an option:")
    print("1. Calculate hashes for text")
    print("2. Check a downloaded file")
```

```
    print("3. Exit")

    choice = input("Enter your choice (1-3): ").strip()

    if choice == "1":
        text = input("\nEnter text to hash: ")
        calculate_text_hashes(text)

    elif choice == "2":
        try:
            check_file_integrity()
        except OSError:
            print("Error: Unable to read the file.")

    elif choice == "3":
        print("\nThank you for using the Hash Calculator!")
        break

    else:
        print("Invalid choice. Please enter 1, 2, or 3.")
```

Important: This program uses SHA-1 instead of SHA-128 because SHA-128 is not a standard algorithm name. If your instructor confirms a different intended algorithm, the code can be adjusted.

---

3. Understanding How the Program Works

Here are the main Python concepts used in the program.

Python concept	Purpose
`import hashlib`	Loads Python's hashing tools.
`import os`	Allows the program to check whether a file exists.
`def`	Defines a reusable function.
`hashlib.md5()`	Calculates an MD5 hash.

Python concept	Purpose
`hashlib.sha1()`	Calculates a SHA-1 hash.
`hashlib.sha256()`	Calculates a SHA-256 hash.
`.hexdigest()`	Converts the hash into a readable hexadecimal string.
`if` and `else`	Allow the program to make decisions.
`while True`	Keeps the menu running until the user exits.
`try` and `except`	Handle certain file-access errors.
`input()`	Accepts information from the user.
`return`	Sends a result back to the part of the program that called a function.

What is hashing?

Hashing converts data into a fixed-length string of characters.

For example, the text:

```
Hello World
```

will produce a different hash than:

```
Hello world
```

Even though only one letter changes from uppercase to lowercase, the resulting hashes will be substantially different.

This property makes hashing useful for detecting changes to downloaded files.

How does the file integrity checker work?

1. An employee downloads a file.
2. The employee obtains the file's original SHA-256 hash from a trusted source.
3. The program calculates the downloaded file's SHA-256 hash.
4. The program compares the calculated hash with the trusted hash.
5. If the hashes match, the file matches the expected contents. If they differ, the file may have changed or become corrupted.

Important: The original hash must come from a trusted source. If an attacker can change both the downloaded file and the published hash, the comparison may not detect the attack.

---

4. Five Test Cases

Your assignment requires five test cases. These tests cover normal operation, file integrity, and error handling.

Test case	Test scenario	Input or action	Expected result
1	Calculate text hashes	Select option 1 and enter `Hello World`	The program displays MD5, SHA-1, and SHA-256 hashes.
2	Verify an unchanged file	Create a text file, calculate its SHA-256 hash, and enter that hash when prompted.	The program reports that the hashes match.
3	Detect a modified file	Calculate a file's hash, modify its contents, and check it against the original hash.	The program warns that the hashes do not match.
4	Enter an invalid hash	Select option 2 and enter a valid file path, followed by `12345` as the expected hash.	The program reports that SHA-256 hashes must contain 64 characters.
5	Check a nonexistent file	Select option 2 and enter a path to a file that does not exist.	The program reports that the file cannot be found.

How to perform the tests

Test 1: Text hashing

- Run the program in Spyder.
- Enter `1` at the menu.
- Enter `Hello World`.
- Verify that three hashes are displayed.

Test 2: Unchanged file

- Create a text file named `testfile.txt`.
- Enter some text, such as `Cybersecurity Test`, and save it.
- Select option 2 and enter the file's full path.
- Copy the calculated SHA-256 hash.
- Paste it as the original hash.
- Verify that the hashes match.

Test 3: Modified file

- Use the same file from Test 2.

- Change its contents to `Cybersecurity Test Modified` and save it.
- Select option 2 again.
- Enter the file's path and the original SHA-256 hash from before the modification.
- Verify that the program reports a mismatch.

Test 4: Invalid hash

- Select option 2 and enter the path to an existing file.
- Enter `12345` when prompted for the original hash.
- Verify that the program rejects the input because it is too short.

Test 5: Missing file

- Select option 2.
- Enter a filename or path that does not exist.
- Verify that the program displays `Error: File not found.`

For each test, record the actual result you observe in Spyder. Do not mark a test as successful until you have run it and confirmed the expected behavior.

---

5. Applying the Prompt → Generate → Understand → Test → Verify → Security Functions → Critical Thinking Model

Your assignment asks you to use this model, so you can document your development process as follows.

Stage	How it applies to this project
Prompt	Identify the problem: employees need a way to detect changes to downloaded files.
Generate	Use AI assistance to develop a Python hash calculator and file integrity checker.
Understand	Review the code and learn how hashing algorithms, functions, file reading, and comparisons work.
Test	Run five test cases covering text hashing, unchanged files, modified files, invalid hashes, and missing files.
Verify	Compare the program's actual output with the expected results. Confirm that changing file contents changes its SHA-256 hash.

Stage	How it applies to this project
Security Functions	Use SHA-256 to check file integrity and warn employees when the calculated hash differs from a trusted hash.
Critical Thinking	Consider whether the original hash is trustworthy, why MD5 and SHA-1 are weaker choices, and whether a matching hash guarantees that a file is safe.

---

6. Security Functions and Critical Thinking

Here are some questions you should be able to answer about your program.

1. Why is SHA-256 useful for file integrity checking?

SHA-256 produces a 256-bit hash, normally represented by 64 hexadecimal characters. A change to the file's contents will ordinarily produce a different hash.

2. Why shouldn't employees rely on MD5 alone?

MD5 is vulnerable to collision attacks, in which different inputs can be constructed to produce the same hash. SHA-1 also has known collision weaknesses. SHA-256 is a stronger choice for integrity verification.

3. Does a matching hash guarantee that a downloaded file is safe?

No. A matching hash indicates that the file matches the contents represented by the trusted hash. The file could still contain malware if the original file was malicious.

4. What happens if the file changes after its hash is calculated?

The program calculates a new hash when the file is checked. If the contents have changed, the new SHA-256 hash will ordinarily differ from the original hash.

5. Why should the original hash come from a trusted source?

If an attacker replaces both the downloaded file and its expected hash, the comparison might still succeed. Employees should obtain hashes through an authenticated, trusted source.

---

7. What to Do Next in Spyder 6

1. Create and save `hash_integrity_checker.py`.
2. Run the program using Spyder's green **Run** button.
3. Complete all five test cases.
4. Record the actual results.
5. Review the code and make sure you can explain the functions and hash comparison.
6. Save your code and any required test documentation for submission.

One final clarification: Before submitting, ask your instructor whether *SHA-128* means SHA-1 or another algorithm. SHA-128 is not a standard SHA algorithm name, and confirming the requirement will help ensure your submission matches the assignment.

my instructor provided some files to us for this assignment too

Absolutely, Mike! Go ahead and **upload the files your instructor provided** here in the chat. You can upload Python files (`.py`), PDFs, Word documents, text files, screenshots, or a ZIP folder containing the assignment materials.

Once you upload them, I'll review the materials and help you build the program around your instructor's actual requirements.

Here's what we'll do:

1. **Review the instructor's files** to understand the assignment instructions and any starter code.
2. **Identify the required hashing algorithms**, including what your instructor means by MD5, SHA-128, and SHA-256.
3. **Use the provided code or templates** rather than replacing them with an entirely different program.
4. **Build the Hash Calculator and File Integrity Checker** in Python for Spyder 6.
5. **Create five test cases** that follow the assignment requirements.
6. **Explain the code step by step** so you understand how it works and can explain it yourself.
7. **Follow the Prompt → Generate → Understand → Test → Verify → Security Functions → Critical Thinking model** in your assignment.

One important thing: Let's look at your instructor's files before making further changes to the code I provided earlier. They may contain starter code, specific requirements, or examples that we need to follow.

Upload the files whenever you're ready, and we'll work through them together.

Uploaded a file Uploaded a file Uploaded a file Uploaded a file Uploaded a file Uploaded a file

Thanks, Mike! I've reviewed the files your instructor provided. **These files give us a much clearer picture of what your assignment requires.** We should build on the instructor's starter code rather than replace it with the simpler program I shared earlier.

Your materials contain a Python file integrity checker, a list of trusted SHA-256 hashes, three files to verify, a security policy, and a Word document explaining how to create test cases.

Let's break down what each file does and how we'll use it.

1. What your instructor provided

File	Purpose	How we'll use it
`file_integrity_checker.py`	Python program that checks file integrity	Use this as our starting code.
`expected_hashes.json`	Stores the original SHA-256 hashes	Compare these hashes against the downloaded files.
`student_records.csv`	Example CSV file	Test whether the checker detects changes to a file.
`security_policy.txt`	Explains the organization's security requirements	Use it to understand the security rules the program must follow.
`app_config.json`	Example application configuration	Test whether the checker recognizes an unchanged file.
`Creating a test Case in CIS 226.docx`	Instructions for creating five hashing test cases	Use its five specified examples to test our hash calculator.

The security policy specifically instructs employees to verify files against trusted SHA-256 hashes, investigate mismatches before using files, and obtain the expected hash manifest from a trusted source. security_policy

There are **two parts to your assignment**:

1. A hash calculator that generates hashes for different inputs.
2. A file integrity checker that verifies downloaded files against trusted hashes.

Your instructor has already provided a starting point for the second part.

---

2. Understanding the Python starter code

Your `file_integrity_checker.py` file already includes several important functions.

A. Importing Python libraries

The program starts with:

python

```
import argparse
import hashlib
import json
from pathlib import Path
```

Here's what each library does:

- `argparse`: Allows the program to accept options when it starts.
- `hashlib`: Calculates cryptographic hashes.
- `json`: Reads the expected hashes from a JSON file.
- `Path`: Helps Python work with file paths.

These are standard Python libraries, so you don't need to install them separately.

B. Calculating a file's SHA-256 hash

The starter code contains this function:

python

```python
def sha256_file(path: Path) -> str:
    digest = hashlib.sha256()

    with path.open("rb") as stream:
        for chunk in iter(lambda: stream.read(1024 * 1024), b""):
            digest.update(chunk)

    return digest.hexdigest()
```

Let's simplify what it does.

1. Creates a SHA-256 hash calculator.
2. Opens the file in binary mode.
3. Reads the file in chunks rather than loading the entire file into memory.
4. Adds each chunk to the hash calculation.
5. Returns the completed hash as a hexadecimal string.

Why does this matter?

Files can be large. Reading them in chunks allows the program to calculate hashes without loading an entire file into memory.

C. Comparing the hashes

Later, the program compares the calculated hash with the expected hash:

```python
if actual.lower() == expected.lower():
    print(f"PASS     {filename}  SHA-256: {actual}")
else:
    print(f"FAIL     {filename}")
```

If the hashes match, the file passes verification.

If the hashes differ, the program reports a failure.

The starter code also detects missing files, invalid hash entries, and certain file-access errors. file_integrity_checker

---

3. Your instructor's expected hashes

Your `expected_hashes.json` file contains three trusted SHA-256 values. I checked them against the three corresponding files you uploaded.

All three files currently match their expected hashes.

File	Verification result
`student_records.csv`	✅ PASS
`security_policy.txt`	✅ PASS
`app_config.json`	✅ PASS

This is a useful first test: the files match the expected hashes provided in your assignment materials.

Remember that a matching hash confirms the file matches the expected contents represented by that hash. It does not prove the file is free from malware.

Your instructor's security policy says to investigate any mismatch before using the file. security_policy

---

4. The five hashing tests from your Word document

Your instructor's document specifies five inputs that you must hash and then check against the values generated by your script. Creating a test Case in CIS 226 Creating a test Case in CIS 226

Here are the five tests.

Test	Input	What it tests
1	`2 + 2 = 5`	Hashing a short string containing numbers and symbols.
2	`Good Morning! How are you!!`	Hashing a normal greeting.
3	The complete song lyrics provided in the assignment	Hashing a longer, multiline text input.
4	`Pi = 3.14159`	Hashing a string containing a mathematical value.
5	`Dark Side of the Moon`	Hashing a short phrase.

Important detail for Test 3

For the third test, use the complete text from your instructor's document.

Preserve the original:

- Capitalization.
- Punctuation.
- Apostrophes.
- Spaces.
- Line breaks.

Even a small difference changes the hash.

The Word document asks you to check each value your script generates against the values obtained from an online hash calculator. Creating a test Case in CIS 226

For consistency, you should use the same exact input when comparing the two calculators.

---

5. What we should change or add

I recommend that we keep the instructor's file integrity checker and add the hash calculator as a separate function or menu option.

The completed program should support these capabilities:

Feature	Status
Calculate SHA-256 hashes for files	Already provided
Compare file hashes against a trusted manifest	Already provided

Feature	Status
Detect missing files	Already provided
Calculate MD5 hashes for text	Needs to be added
Calculate SHA-1 hashes for text	Needs to be added if SHA-128 means SHA-1
Calculate SHA-256 hashes for text	Needs to be added
Run the five required text tests	Needs to be tested
Verify the three provided files	Ready to test
Detect changes to a file	Ready to test by modifying a copy

One question remains: Your assignment says MD5, SHA-128, and SHA-256. SHA-128 is not a standard SHA algorithm name, so we shouldn't assume that SHA-1 is what your instructor intended.

The starter code specifically uses SHA-256 for file verification, which is consistent with the supplied expected hashes.

---

6. My recommendation for your next step

Let's build the finished version around the files you received.

We'll:

1. Keep the existing file integrity checker.
2. Add a simple text hash calculator supporting the algorithms your instructor requires.
3. Create a menu so you can choose between calculating hashes and verifying files.
4. Test the program with all five inputs from the Word document.
5. Verify the three supplied files.
6. Test what happens when a file is modified.
7. Document the results using the Prompt → Generate → Understand → Test → Verify → Security Functions → Critical Thinking model.

This approach will help you meet the assignment requirements while understanding the Python code rather than just copying it.

Before we finalize the code, can you confirm one thing? Did your instructor specifically say **SHA-128**, or could the assignment actually say **SHA-1**? That will determine which algorithms we implement in the hash calculator.

help me with the next step

Absolutely, Mike! Let's take it one step at a time so you can complete the assignment in **Spyder 6** without getting overwhelmed.

Step 1: Run the provided file integrity checker

Before adding anything new, let's get your instructor's starter program running. This will help you understand how the file integrity checker works.

1. Put the assignment files in one folder

Create a folder on your computer called:

`CIS 226 - Hash Calculator`

Place these five files inside it:

- `file_integrity_checker.py`
- `expected_hashes.json`
- `student_records.csv`
- `security_policy.txt`
- `app_config.json`

Important: Keep the filenames exactly as shown. The Python script uses those names to locate and verify the files.

2. Open the Python script in Spyder 6

1. Open Spyder 6.
2. Select **File** → **Open**.
3. Find your `CIS 226 - Hash Calculator` folder.
4. Open `file_integrity_checker.py`.
5. Look through the code without changing anything yet.

The program uses Python's `hashlib` library to calculate a SHA-256 hash for each file. It then compares the calculated hash against the expected hash saved in `expected_hashes.json`.

3. Run the program

Click the green **Run** button or press **F5**.

Look at the IPython Console at the bottom of Spyder. If the files are in the correct folder and haven't changed, you should see output similar to this:

```
text

PASS     student_records.csv  SHA-256: [calculated hash]
PASS     security_policy.txt  SHA-256: [calculated hash]
PASS     app_config.json  SHA-256: [calculated hash]

Result: 3/3 files verified
```

The actual hashes will be long strings of letters and numbers.

What does this mean?

- **PASS:** The calculated hash matches the expected hash.
- **FAIL:** The file's calculated hash doesn't match the expected hash.
- **MISSING:** The program couldn't find the file.
- **ERROR:** The program encountered a problem reading the manifest or checking a file.

The provided manifest contains expected hashes for three files, so a successful run should verify all three. file_integrity_checker

---

Step 2: Understand what we're building next

Your assignment calls for a hash calculator using MD5, SHA-128, and SHA-256, along with a file integrity checker.

We'll build on the provided script rather than starting over.

Our finished program will have two main features:

Feature	What it does
Hash Calculator	Calculates hashes for text using the required algorithms.
File Integrity Checker	Checks whether downloaded files match their expected SHA-256 hashes.

One small detail: **SHA-128 isn't a standard algorithm name supported by Python's `hashlib`.** SHA-1 produces a 160-bit hash, while SHA-256 produces a 256-bit hash. We'll need to confirm whether your instructor intended SHA-1 before finalizing that part.

Your instructor's test-case document also gives you five pieces of text to hash and compare against an online calculator. Creating a test Case in CIS 226

Let's start with just one thing.

Try Step 1 and tell me what happens when you press F5 in Spyder:

1. **It works** — you see `Result: 3/3 files verified`.
2. **I get an error** — copy and paste the error message here.
3. **I'm stuck opening the files** — I'll walk you through getting them into Spyder.

Once the starter program runs, we'll add the hash calculator together, one small section of Python at a time.

1

Awesome, Mike! 🎉 That means your file integrity checker is working, and all three files passed verification.

Now let's move on to **Step 2: Building your Hash Calculator**. We'll keep the existing file checker and add a feature that calculates hashes for text.

Step 2A: Add the Hash Calculator

Open `file_integrity_checker.py` in Spyder 6.

At the top of the file, look for the existing imports. You should see something similar to:

python

```python
import argparse
import hashlib
import json
from pathlib import Path
```

We can use the existing `hashlib` library, so you don't need to install anything extra.

1. Add this function

Find the section just before `def main():` and add the following code above it:

python

```python
def calculate_text_hashes():
    """Calculate hashes for text entered by the user."""

    text = input("Enter text to hash: ")

    # Convert the text into bytes for hashing
```

```
    text_bytes = text.encode("utf-8")

    # Calculate the hashes
    md5_hash = hashlib.md5(text_bytes).hexdigest()
    sha1_hash = hashlib.sha1(text_bytes).hexdigest()
    sha256_hash = hashlib.sha256(text_bytes).hexdigest()

    # Display the results
    print("\nHash Results")
    print("-" * 40)
    print("MD5:    ", md5_hash)
    print("SHA-1:  ", sha1_hash)
    print("SHA-256:", sha256_hash)
```

Why are we using SHA-1? Your assignment mentions SHA-128, but that isn't a standard algorithm name in Python. I'm using SHA-1 temporarily so we can test the program. We'll confirm the intended algorithm before submitting.

2. Test the new function

At this point, we need to call the function so Python runs it.

For now, the easiest way to test it is to temporarily add these two lines at the very bottom of the file:

python

```
if __name__ == "__main__":
    calculate_text_hashes()
```

Important: If your existing script already has an `if __name__ == "__main__":` section, don't add a second one yet. Tell me what that section looks like, and we'll connect the function correctly.

3. Run the test

Press **F5** in Spyder.

When prompted, enter:

text

```
Dark Side of the Moon
```

The program should display three hash values: one for MD5, one for SHA-1, and one for SHA-256.

You can then test the other four text entries in your instructor's test-case document.

What the code does

Code	Purpose
`input()`	Lets you enter text.
`.encode("utf-8")`	Converts text into bytes that the hashing functions can process.
`hashlib.md5()`	Calculates an MD5 hash.
`hashlib.sha1()`	Calculates a SHA-1 hash.
`hashlib.sha256()`	Calculates a SHA-256 hash.
`.hexdigest()`	Returns the hash as a readable string of hexadecimal characters.

Remember: hashing helps detect changes, but MD5 and SHA-1 are not recommended for modern security-sensitive integrity checks. Your existing SHA-256 file checker is the stronger choice.

Your next step

Before we combine everything, **look at the bottom of `file_integrity_checker.py`**.

Do you see this?

```python
if __name__ == "__main__":
    main()
```

Reply with either:

- **A:** Yes, I see it.
- **B:** No, my code looks different.

I'll guide you through the next step based on your answer so we don't accidentally break the working file checker.

b

No problem, Mike! 👍 We'll figure out what your script looks like before changing anything.

Step 1: Check the bottom of your Python file

In Spyder 6:

1. Open `file_integrity_checker.py`.
2. Press **Ctrl + End** to jump to the bottom of the file.
3. Look at the last 10–15 lines of code.

You might see something like:

```python
if __name__ == "__main__":
    main()
```

Or your file might end with something different.

Step 2: Tell me what you see

Which option best describes the bottom of your file?

- **A.** It ends with `main()` but doesn't have the `if __name__` line.
- **B.** It ends with `main()` and has the `if __name__` line.
- **C.** It ends with something else.
- **D.** I'm not sure what I'm looking at.

If you're unsure, you can **copy and paste the last 10–15 lines of your code here**. I'll show you exactly what to change without breaking the file checker that is already working.

```
if __name__ == "__main__":
    raise SystemExit(main())
```

Perfect, Mike! That's actually exactly what we want. 👍 Your existing script uses:

```python
if __name__ == "__main__":
    raise SystemExit(main())
```

Don't change or delete those lines. They're responsible for running your existing file integrity checker.

Instead, we'll modify the program so you can choose between calculating text hashes and checking file integrity.

Step 3: Add a menu to your program

We'll make a small change so your program offers two options:

1. Calculate hashes for text.
2. Check the integrity of your files.

1. Add the hash calculator function

In `file_integrity_checker.py`, find the line that says:

```python
def main():
```

Above that line, paste this function:

```python
def calculate_text_hashes():
    """Calculate MD5, SHA-1, and SHA-256 hashes for text."""

    text = input("\nEnter the text you want to hash: ")
    text_bytes = text.encode("utf-8")

    md5_hash = hashlib.md5(text_bytes).hexdigest()
    sha1_hash = hashlib.sha1(text_bytes).hexdigest()
    sha256_hash = hashlib.sha256(text_bytes).hexdigest()

    print("\n--- Hash Results ---")
    print("MD5:    ", md5_hash)
    print("SHA-1:  ", sha1_hash)
    print("SHA-256:", sha256_hash)
```

This function uses the `hashlib` library that's already imported in your script.

2. Modify the main function

Before we change `main()`, I want to make sure we preserve your instructor's existing code.

Please copy and paste your entire current `main()` function here, starting with:

```python
```

```
def main():
```

and ending just before:

```
python

if __name__ == "__main__":
```

I'll modify it to add the menu while keeping the existing file checker intact.

One important note: Your assignment says SHA-128, but the standard Python hashing algorithms include SHA-1 and SHA-256—not SHA-128. We'll use SHA-1 temporarily and confirm what your instructor intended before finalizing your submission.

```
def main() -> int:
    parser = argparse.ArgumentParser(description="Check SHA-256 download integrity")
    parser.add_argument("--directory", type=Path, default=Path(__file__).resolve().parent,
                        help="Folder containing downloaded files (default: script folder)")
    parser.add_argument("--manifest", type=Path, default=Path(__file__).resolve().parent / "expected_hashes.json",
                        help="Trusted JSON file of expected SHA-256 hashes")
    args = parser.parse_args()

    try:
        manifest = json.loads(args.manifest.read_text(encoding="utf-8"))
        if manifest.get("algorithm") != "SHA-256" or not isinstance(manifest.get("files"), dict):
            raise ValueError("Manifest must contain algorithm SHA-256 and a files object")
    except (OSError, ValueError, json.JSONDecodeError) as error:
        print(f"ERROR reading manifest: {error}")
        return 2

    failed = 0
```

```
    for filename, expected in manifest["files"].items():
        if (not isinstance(filename, str) or Path(filename).name !=
filename
                or not isinstance(expected, str) or len(expected) != 64
                or any(c not in "0123456789abcdefABCDEF" for c in
expected)):
            print(f"ERROR: Invalid manifest entry for {filename!r}")
            return 2
        path = args.directory / filename
        if not path.is_file():
            print(f"MISSING  {filename}")
            failed += 1
            continue
        try:
            actual = sha256_file(path)
        except OSError as error:
            print(f"ERROR    {filename}: {error}")
            failed += 1
            continue
        if actual.lower() == expected.lower():
            print(f"PASS     {filename}  SHA-256: {actual}")
        else:
            print(f"FAIL     {filename}")
            print(f"         Expected: {expected.lower()}")
            print(f"         Actual:   {actual}")
            failed += 1

    print(f"\nResult: {len(manifest['files']) -
failed}/{len(manifest['files'])} files verified")
    return 1 if failed else 0
```

Great, Mike! You've given me exactly what I needed. Your existing `main()` function handles file integrity checking, so **we'll keep that functionality and add a menu without deleting your instructor's code**.

We'll make this change in small steps.

Step 3: Add the hash calculator menu

1. Add the new function

In Spyder, find the line immediately above `def main() -> int:`.

Paste this code there:

```python
def calculate_text_hashes():
    """Calculate MD5, SHA-1, and SHA-256 hashes for text."""

    text = input("\nEnter the text you want to hash: ")
    text_bytes = text.encode("utf-8")

    md5_hash = hashlib.md5(text_bytes).hexdigest()
    sha1_hash = hashlib.sha1(text_bytes).hexdigest()
    sha256_hash = hashlib.sha256(text_bytes).hexdigest()

    print("\n--- Hash Results ---")
    print("MD5:     ", md5_hash)
    print("SHA-1:   ", sha1_hash)
    print("SHA-256: ", sha256_hash)
```

This function takes text you enter and calculates three hashes.

2. Add a menu to `main()`

We'll make a small change to the beginning of your existing `main()` function.

Find these lines:

```python
def main() -> int:
    parser = argparse.ArgumentParser(description="Check SHA-256 download
integrity")
```

Replace just those two lines with this code:

```python
def main() -> int:
    print("\n=== Cybersecurity Hash Calculator ===")
    print("1. Calculate text hashes")
    print("2. Check file integrity")

    choice = input("Choose an option (1 or 2): ")
```

```
    if choice == "1":
        calculate_text_hashes()
        return 0

    elif choice == "2":
        print("\nStarting file integrity checker...\n")

    else:
        print("Invalid choice. Please run the program again and select 1 or
2.")
        return 2

    parser = argparse.ArgumentParser(description="Check SHA-256 download
integrity")
```

Leave everything else in your original `main()` function unchanged. In particular, keep all the code that reads the manifest, calculates file hashes, and reports `PASS`, `FAIL`, or `MISSING`.

Also, leave the bottom of your file as it is:

python

```
if __name__ == "__main__":
    raise SystemExit(main())
```

Step 4: Test the new menu

Save your file with **Ctrl + S**, then press **F5**.

You should see:

text

```
=== Cybersecurity Hash Calculator ===
1. Calculate text hashes
2. Check file integrity
Choose an option (1 or 2):
```

Test A: Calculate a hash

Enter:

text

```
1
```

When prompted for text, enter:

```
text

Dark Side of the Moon
```

Your program should display MD5, SHA-1, and SHA-256 hash values.

Test B: Check file integrity

Run the program again and enter:

```
text

2
```

The existing file integrity checker should run as before, verifying the three files.

Step 5: One important assignment detail

Your assignment specifies **MD5, SHA-128, and SHA-256**. The code above uses SHA-1 as a temporary substitute because SHA-128 is not a standard algorithm name in Python's `hashlib`.

Before submitting, we should confirm what your instructor means by SHA-128 rather than silently treating SHA-1 as equivalent.

Your next step

Make the changes above and press **F5**.

Tell me which result you get:

- **A:** The menu appears, and option 1 calculates the hashes.
- **B:** The menu appears, but option 1 produces an error.
- **C:** I get a syntax or indentation error.
- **D:** Option 1 works, but option 2 no longer verifies the files.

We'll troubleshoot whatever happens next.

A

Nice work, Mike! 🎉 You've successfully added the text hash calculator to your existing program in Spyder 6.

Now let's move on to **Step 4: Test your program using the five test cases from your instructor's document.**

Step 4: Test your hash calculator

Your instructor wants you to hash five pieces of text and check the values your program generates against an online hash calculator. Creating a test Case in CIS 226

We'll test them one at a time.

Test Case 1

In Spyder, press **F5** to run the program.

When the menu appears, enter `1`.

When prompted for text, enter:

```text
2 + 2 = 5
```

Your program should display three hash values:

- MD5
- SHA-1
- SHA-256

Write down the three values or copy them into a text file. We'll compare them with the results from an online calculator.

Test Case 2

Run the program again and enter:

```text
Good Morning!  How are you!!
```

Important: The instructor's text appears to contain two spaces between the exclamation mark and `How`. Make sure you preserve the spacing exactly.

Record all three hash values.

Test Case 3

Use the full lyrics specified in your instructor's document for *A Hard Day's Night*.

Since this test contains multiple lines, you'll need to modify the function to accept multiline text or temporarily use a multiline string. We can handle that after the first two tests.

Test Cases 4 and 5

Enter the following separately:

Test 4

```text
Pi = 3.14159
```

Test 5

```text
Dark Side of the Moon
```

These are also listed in your instructor's test-case document. Creating a test Case in CIS 226

Step 5: Verify your results

For each test case, compare your Python program's hashes with the results from an online hash calculator.

The results for the same algorithm and **exact same input** should match.

Remember:

- Capitalization matters.
- Spaces matter.
- Punctuation matters.
- Line breaks matter for multiline text.

If even one character changes, the hash will usually be completely different.

One thing to confirm before submitting

Your assignment asks for **SHA-128**, but our program currently calculates SHA-1 instead. These are not the same algorithm. We'll need to clarify the instructor's intended algorithm before finalizing your code.

Your next step

Let's keep this simple for now.

Run Test Case 1 (`2 + 2 = 5`) and tell me when you've recorded the MD5, SHA-1, and SHA-256 results.

Then I'll help you verify the values and build your five-test-case results table for the assignment.

MD5: a2f0a1d17ab98af5537e29b5f07ce1a9
SHA-1: 0a04829748e02416ebf978d69358b14db94735c5
SHA-256: a2d9b3b0ec334d3945839fd6f505d952968bd12087a2900a73fef084a78806da

Great job, Mike! 🎉 You've completed your first hash test case, and you've recorded all three results. Let's organize them so you can use them in your assignment.

Step 1: Record Test Case 1

Your input was `2 + 2 = 5`.

Algorithm	Your calculated hash
MD5	`a2f0a1d17ab98af5537e29b5f07ce1a9`
SHA-1	`0a04829748e02416ebf978d69358b14db94735c5`
SHA-256	`a2d9b3b0ec334d3945839fd6f505d952968bd12087a2900a73fef084a78806da`

These are the values your program generated. We still need to compare them with an independent hash calculator to verify that they match.

Step 2: Verify your hashes

Open an online hashing tool, such as Online Hash Tools, and enter the exact text:

`2 + 2 = 5`

Calculate the MD5, SHA-1, and SHA-256 hashes, then compare them with your table.

Important: Make sure there are no extra spaces before or after the text.

If all three hashes match, Test Case 1 passes verification.

Step 3: Move on to Test Case 2

Let's keep working through the instructor's five test cases.

Run your program again by pressing **F5**, select option `1`, and enter this text exactly:

```
text
Good Morning!  How are you!!
```

There are **two spaces** between `!` and `How`.

Copy the three resulting hashes and send them to me. I'll add them to our results table and help you continue through the remaining test cases.

One note for your final assignment: Your instructor specifies SHA-128, while our current program uses SHA-1 as a temporary substitute. We should clarify that requirement before calling the program complete.

this is what came up on the online calculator for the 2 + 2 = 5 MD5:a88e969726a9a70d150492c9e1397378 SHA-1:c366599b0d774f81430c7ab75d01c7b865ce3082 SHA-256:81ee9ec29728006a80227e319b91de189d34d9db8c1fc42329f96e17f3735ef9

Good catch, Mike! This is exactly why your instructor wants you to verify your program against an independent hash calculator. Let's troubleshoot the difference before moving on.

Step 1: Compare the results

For the exact text `2 + 2 = 5`, these are the hashes your Python program generated:

Algorithm	Python result	Online calculator result	Match?
MD5	`a2f0a1d17ab98af5537e29b5f07ce1a9`	`a88e969726a9a70d150492c9e1397378`	❌
SHA-1	`0a04829748e02416ebf978d69358b14db94735c5`	`c366599b0d774f81430c7ab75d01c7b865ce3082`	❌
SHA-256	`a2d9b3b0ec334d3945839fd6f505d952968bd12087a2900a73fef084a78806da`	`81ee9ec29728006a80227e319b91de189d34d9db8c1fc42329f96e17f3735ef9`	❌

I independently recalculated the hashes for the exact text `2 + 2 = 5`, and **your Python program's three results are correct for that input**.

This suggests the online calculator is hashing something different, or one of its settings is affecting the input.

Step 2: Check the online calculator

Let's make sure both programs are hashing exactly the same text.

1. Return to the online calculator.
2. Clear its input field completely.
3. Make sure you're using its **text input** option, not a file or another input mode.
4. Type the following manually, without quotation marks:

   ```
   2 + 2 = 5
   ```

5. Make sure there are no spaces before or after the text.
6. Calculate the MD5 hash again.

Your MD5 result should be:

```text
a2f0a1d17ab98af5537e29b5f07ce1a9
```

If it matches, repeat the process for SHA-1 and SHA-256.

Why does this matter?

Hashing is sensitive to the exact input. Even a single extra space or a hidden character can produce a completely different hash.

For example, these are three different inputs:

- `2 + 2 = 5`
- `2 + 2 = 5 ` — an extra space at the end
- `2 + 2 = 5` followed by a newline

They produce different hashes.

Step 3: Tell me what happens

Try the MD5 calculation again and let me know:

A. The online calculator now matches my Python result.

B. The online calculator still gives me `a88e969726a9a70d150492c9e1397378`.

C. I can't find the text-input or algorithm settings.

We'll resolve this before recording Test Case 1 as verified.

now the calculator shows MD5:
8b82f7dca114cbc6e8b8a5371171e541

No worries, Mike! We're getting closer. The new MD5 result, `8b82f7dca114cbc6e8b8a5371171e541`, still doesn't match the hash generated by your Python program.

For the exact text `2 + 2 = 5`, the correct MD5 hash is:

```text
a2f0a1d17ab98af5537e29b5f07ce1a9
```

Let's check one thing before changing any code.

Step 1: Check the online calculator's input

1. Go back to the online calculator.
2. Completely clear its input field.
3. Type this exact text, without quotation marks:

   ```
   2 + 2 = 5
   ```

4. Make sure the calculator is set to **MD5**.
5. Calculate the hash again.

Step 2: If it still doesn't match

Tell me **which website you're using for the online calculator**. You can send me its name or paste its web address.

I want to make sure we're comparing the same input and algorithm rather than changing working Python code unnecessarily.

Remember: Your original Python result is correct for the exact text shown above, so don't modify your script yet!

a2f0a1d17ab98af5537e29b5f07ce1a9

Perfect, Mike! 🎉 Your online calculator now matches your Python program for Test Case 1.

✅ Test Case 1: Verified

Input: `2 + 2 = 5`

Algorithm	Hash result	Status
MD5	`a2f0a1d17ab98af5537e29b5f07ce1a9`	✅ Match
SHA-1	`0a04829748e02416ebf978d69358b14db94735c5`	⏳ Needs verification
SHA-256	`a2d9b3b0ec334d3945839fd6f505d952968bd12087a2900a73fef084a78806da`	⏳ Needs verification

Only the MD5 result has been independently verified so far. Let's check the other two algorithms before marking the entire test case complete.

Step 1: Verify SHA-1

On the online calculator, select **SHA-1** and enter the same text:

`2 + 2 = 5`

Compare the result against your Python output:

text

```
0a04829748e02416ebf978d69358b14db94735c5
```

Step 2: Verify SHA-256

Select **SHA-256** and enter the same text again.

Compare the result against:

text

```
a2d9b3b0ec334d3945839fd6f505d952968bd12087a2900a73fef084a78806da
```

Step 3: Move to Test Case 2

Once those checks are complete, run your Python program again by pressing **F5**, select option 1, and enter:

```text
Good Morning!  How are you!!
```

Remember that there are two spaces between `!` and `How`.

Send me the results, and we'll continue building your five-test-case results table.

You're doing well! You're learning how to test code against an independent reference rather than simply assuming the program works. That's an important skill in cybersecurity.

0a04829748e02416ebf978d69358b14db94735c5

Great, Mike! ✅ Your SHA-1 result matches your Python program.

Test Case 1 — Updated Results

Input: `2 + 2 = 5`

Algorithm	Result	Verification
MD5	`a2f0a1d17ab98af5537e29b5f07ce1a9`	✅ Match
SHA-1	`0a04829748e02416ebf978d69358b14db94735c5`	✅ Match
SHA-256	`a2d9b3b0ec334d3945839fd6f505d952968bd12087a2900a73fef084a78806da`	⏳ To verify

Next step: Verify SHA-256

On your online calculator:

1. Select **SHA-256**.
2. Enter `2 + 2 = 5` again.
3. Compare its result with this value from your Python program:

```text
a2d9b3b0ec334d3945839fd6f505d952968bd12087a2900a73fef084a78806da
```

Send me the SHA-256 result from the online calculator. Once it matches, we'll mark Test Case 1 complete and move on to Test Case 2!

a2d9b3b0ec334d3945839fd6f505d952968bd12087a2900a73fef084
a78806da

Excellent, Mike! 🎉 All three hashes for Test Case 1 have now been verified against the online calculator.

✅ Test Case 1: Complete

Input: `2 + 2 = 5`

Algorithm	Hash	Verification
MD5	`a2f0a1d17ab98af5537e29b5f07ce1a9`	✅ Match
SHA-1	`0a04829748e02416ebf978d69358b14db94735c5`	✅ Match
SHA-256	`a2d9b3b0ec334d3945839fd6f505d952968bd12087a2900a73fef084a78806da`	✅ Match

What you've learned: Your Python program generates the same hashes as the independent online calculator when both use the same text and algorithm. This verifies your results for this test case.

Next: Test Case 2

Let's continue with the second test from your instructor's document.

1. Run your program

In Spyder, press **F5**, then select option `1`.

2. Enter this exact text

```text
Good Morning!  How are you!!
```

Important: There are two spaces between the exclamation mark and `How`.

3. Record the three hashes

Your program should display:

- MD5

- SHA-1
- SHA-256

Copy those three values and send them to me.

We'll verify each one against the online calculator, just as we did for Test Case 1.

You're making good progress! We'll work through all five tests before preparing your final results table. One reminder: your assignment says SHA-128, while the program currently uses SHA-1. We'll need to clarify that discrepancy before your final submission.

MD5: 74720f72161244a230813149745744d7
SHA-1: 88885775955b17d095d1ee57976829beb3de7acc
SHA-256:
a41dbea4bcb2be676ec1a18c1272b1bc8bb95d65894f96a670e655df5e61f398

Perfect, Mike! I've added your results for Test Case 2. Now we need to verify them against the online calculator.

Test Case 2: Results

Input: `Good Morning! How are you!!`

Algorithm	Python-generated hash	Verification
MD5	`74720f72161244a230813149745744d7`	⏳ Pending
SHA-1	`88885775955b17d095d1ee57976829beb3de7acc`	⏳ Pending
SHA-256	`a41dbea4bcb2be676ec1a18c1272b1bc8bb95d65894f96a670e655df5e61f398`	⏳ Pending

Next: Verify Test Case 2

Open your online hash calculator and enter the exact text:

`Good Morning! How are you!!`

Remember:

- Two spaces between `!` and `How`.
- One space between `Good` and `Morning`.

- Two exclamation marks at the end.

Check each algorithm separately.

Start with MD5.

Your expected result from Python is:

```
text

74720f72161244a230813149745744d7
```

Send me the MD5 result from the online calculator, and we'll verify it before moving to SHA-1 and SHA-256.

I got e4a50aa23aa3381e76015ea70eef0250 on md5 hash

Good find, Mike! We figured out what's happening. Your online calculator is hashing a version of the text with **one space instead of two**.

I checked both possibilities:

Input	MD5 hash
`Good Morning! How are you!!` (two spaces)	`74720f72161244a230813149745744d7`
`Good Morning! How are you!!` (one space)	`e4a50aa23aa3381e76015ea70eef0250`

Your online result matches the **one-space version** exactly!

What to do next

Your instructor's test document specifies two spaces between `!` and `How`, so we should preserve those two spaces.

1. Go back to the online calculator.
2. Clear the input field.
3. Copy and paste this exact text:

   ```
   Good Morning! How are you!!
   ```

4. Select MD5 and calculate again.

Your expected MD5 result should be:

```
text

74720f72161244a230813149745744d7
```

Lesson for your assignment: Even one extra or missing space changes the resulting hash. This demonstrates why exact file and data integrity verification matters.

Send me the updated MD5 result when you have it!

74720f72161244a230813149745744d7

Perfect! ✅ Your MD5 hash now matches your Python program.

Test Case 2 — MD5 Verified

Input: `Good Morning! How are you!!`

Algorithm	Python hash	Status
MD5	`74720f72161244a230813149745744d7`	✅ Match
SHA-1	`88885775955b17d095d1ee57976829beb3de7acc`	⏳ Pending
SHA-256	`a41dbea4bcb2be676ec1a18c1272b1bc8bb95d65894f96a670e655df5e61f398`	⏳ Pending

Next: Verify SHA-1

On your online calculator:

1. Select SHA-1.
2. Keep the exact same text (including the two spaces).
3. Calculate the hash.

Your expected result is:

```
text

88885775955b17d095d1ee57976829beb3de7acc
```

Send me the result, and we'll verify it before moving on to SHA-256.

88885775955b17d095d1ee57976829beb3de7acc

Excellent, Mike! ✅ Your SHA-1 result matches your Python program.

Test Case 2 — Updated Results

Input: `Good Morning! How are you!!`

Algorithm	Verification
MD5	✅ Match
SHA-1	✅ Match
SHA-256	⏳ Pending

Next: Verify SHA-256

On your online calculator:

1. Select **SHA-256**.
2. Keep the exact same input, including the two spaces.
3. Calculate the hash.

Your expected result is:

```text
a41dbea4bcb2be676ec1a18c1272b1bc8bb95d65894f96a670e655df5e61f398
```

Send me the SHA-256 result when you have it.

Once that matches, we'll move on to Test Case 3!

You're doing great. We're documenting each test as we go so you'll have the verification results ready for your assignment.

a41dbea4bcb2be676ec1a18c1272b1bc8bb95d65894f96a670e655df5e61f398

Awesome, Mike! 🎉 All three algorithms have now been independently verified for Test Case 2.

✅ Test Case 2: Complete

Input: `Good Morning! How are you!!`

Algorithm	Verification
MD5	✅ Match
SHA-1	✅ Match
SHA-256	✅ Match

Your Python program successfully generated the same hashes as the online calculator.

Progress so far

Test Case	Input	Status
1	`2 + 2 = 5`	✅ 3/3 verified
2	`Good Morning! How are you!!`	✅ 3/3 verified
3	Lyrics from instructor's document	⏳ Not started
4	`Pi = 3.14159`	⏳ Not started
5	`Dark Side of the Moon`	⏳ Not started

Next: Test Case 3

Your instructor's third test involves the lyrics to *A Hard Day's Night*.

Because this test contains multiple lines, we'll need to make a small improvement to your Python program so it can accept multiline text.

Here's what I'd like to do next:

We'll modify the hash calculator to let you paste multiple lines of text instead of being limited to a single line.

That will also make your program more useful for hashing longer text.

Before we continue: Do you still have the original instructor document, `Creating a test Case in CIS 226.docx`, available?

We can use its exact test input so your results match the assignment.

yes

Worked for a few seconds

Perfect! I found your original **Creating a test Case in CIS 226.docx** document.

Your instructor's third test case is a multiline passage from *A Hard Day's Night*. We'll use the exact text from the document, preserving punctuation, capitalization, and line breaks.

Step 4: Prepare your Python program for multiline text

Currently, your hash calculator uses:

```python
text = input("\nEnter the text you want to hash: ")
```

The problem is that `input()` only accepts one line at a time.

We'll change it so you can paste multiple lines.

1. Find your `calculate_text_hashes()` function

Replace the entire function with this:

```python
def calculate_text_hashes():
    """Calculate MD5, SHA-1, and SHA-256 hashes for text."""

    print("\nEnter your text below.")
    print("Type END on a new line when finished.")

    lines = []

    while True:
        line = input()

        if line == "END":
            break
```

```
        lines.append(line)

    text = "\n".join(lines)
    text_bytes = text.encode("utf-8")

    md5_hash = hashlib.md5(text_bytes).hexdigest()
    sha1_hash = hashlib.sha1(text_bytes).hexdigest()
    sha256_hash = hashlib.sha256(text_bytes).hexdigest()

    print("\n--- Hash Results ---")
    print("MD5:     ", md5_hash)
    print("SHA-1:   ", sha1_hash)
    print("SHA-256: ", sha256_hash)
```

2. How the new function works

We've introduced three new concepts:

Code	Purpose
`lines = []`	Creates an empty list to store your text.
`while True:`	Keeps accepting lines until told to stop.
`lines.append(line)`	Adds each line to the list.
`"\n".join(lines)`	Combines the lines while preserving line breaks.
`if line == "END":`	Stops accepting input when you type END.

3. Test the new function

1. Save your program with **Ctrl + S**.
2. Press **F5**.
3. Select option 1.
4. Paste the multiline text from Test Case 3 in your instructor's document.
5. On a new line, type:

```
text

END
```

6. Press Enter.

Your program should generate all three hashes.

Important: Don't include the `END` marker in the text being hashed. Also, preserve the original text's punctuation and line breaks. The document's formatting may affect the exact input, so we'll use the source text rather than retyping it from memory.

What I need from you next

Once you've pasted the text and run the program, send me the three results:

- MD5
- SHA-1
- SHA-256

We'll verify them against the online calculator just like we did with the first two test cases.

We're making good progress! After Test Case 3, only two more test cases remain.

```
--- Hash Results ---
MD5:      864b0a057e3cf9b1c5cdb7a3e1d12b18
SHA-1:    a3b839313e70b8b5308c3402fe33773b79d35d33
SHA-256:
c0a69e523f131130b7d8ba1bcf8561599d8812cc7296a8abcd41569d
aea692b7
```

Great work, Mike! You've successfully run your first multiline hash test. All three hashes have been generated.

Test Case 3: Results

Input: Lyrics from *A Hard Day's Night* (instructor's document).

Algorithm	Python-generated hash	Verification
MD5	`864b0a057e3cf9b1c5cdb7a3e1d12b18`	⏳ Pending
SHA-1	`a3b839313e70b8b5308c3402fe33773b79d35d33`	⏳ Pending
SHA-256	`c0a69e523f131130b7d8ba1bcf8561599d8812cc7296a8abcd41569daea692b7`	⏳ Pending

Step 1: Verify MD5

Open your online hash calculator.

1. Select MD5.
2. Copy the complete lyrics from your instructor's document.
3. Paste them into the calculator's text input.
4. Preserve all line breaks, spaces, and punctuation.
5. Calculate the hash.

Your expected MD5 result is:

```text
864b0a057e3cf9b1c5cdb7a3e1d12b18
```

Important: Make sure you don't include the word `END` in the online calculator. Also, your Python program joins lines with `\n` and does not automatically add a newline after the final line. The online input needs to match that behavior.

Send me the MD5 result from the online calculator, and we'll verify it.

We're now halfway through your five test cases!

20ad50ea1be907bc11cfad1777351e65

Thanks, Mike! Your online calculator returned this MD5 hash:

```text
20ad50ea1be907bc11cfad1777351e65
```

However, it doesn't match the hash generated by your Python program:

```text
864b0a057e3cf9b1c5cdb7a3e1d12b18
```

Test Case 3 — MD5 Comparison

Source	MD5 result
Python program	`864b0a057e3cf9b1c5cdb7a3e1d12b18`
Online calculator	`20ad50ea1be907bc11cfad1777351e65`
Status	❌ Mismatch

Let's troubleshoot before moving on

Multiline text is particularly sensitive to formatting. The difference could be caused by:

- Extra spaces between words.
- Different line breaks.
- An extra blank line.
- A newline at the end of the text.
- Different punctuation or capitalization.

One question: Did you copy the lyrics directly from your instructor's Word document, or did you copy them from the text displayed in Spyder?

I want to make sure we're comparing identical inputs before changing anything in your program.

The text in spyder

That helps! Since you copied the text from Spyder, let's check whether the online calculator is receiving exactly the same text as your program.

One important detail: **your Python program joins each line with `\n`**, so the line breaks are included in the hash.

Let's try this:

1. Go back to Spyder and look at the lyrics you entered.
2. Check for any blank lines between verses.
3. Make sure there are no extra spaces at the beginning or end of lines.
4. Copy the lyrics again, including the same line breaks, into the online MD5 calculator.
5. Generate the MD5 hash again.

Send me the new MD5 result. We'll get Test Case 3 matching before moving on to SHA-1 and SHA-256.

this is off the spyder code: It's been a hard day's night
And I've been working like a dog
It's been a hard day's night
I should be sleeping like a log
But when I get home to you
I find the things that you do
Will make me feel alright
You know I work all day
To get you money, to buy you things
And it's worth it just to hear you say
You're gonna give me everything
So why on earth should I moan
'Cause when I get you alone
You know I feel OK
When I'm home
Everything seems to be right
When I'm home
Feeling you holding me tight
Tight, yeah
It's been a hard day's night
And I've been working like a dog
It's been a hard day's night
I should be sleeping like a log
But when I get home to you
I find the things that you do
Will make me feel alright, owww!
So why on earth should I moan
'Cause when I get you alone
You know I feel OK
When I'm home
Everything seems to be right
When I'm home
Feeling you holding me tight
Tight, yeah
Mmm, it's been a hard day's night
And I've been working like a dog
It's been a hard day's night
I should be sleeping like a log

But when I get home to you
I find the things that you do
Will make me feel alright
You know I feel alright
You know I feel alright

Perfect! Now we have the exact text you're entering into Spyder. I can see a few things worth checking.

Your Python code uses:

```python
text = "\n".join(lines)
```

This means every line is separated by a newline, with no extra newline added after the final line.

Let's verify the MD5 again

1. Copy the lyrics you just sent me.
2. Paste them into the online MD5 calculator.
3. Make sure you don't include anything before the first line or after the last line.
4. Generate the hash.

Your Python program generated:

```text
864b0a057e3cf9b1c5cdb7a3e1d12b18
```

Your previous online result was:

```text
20ad50ea1be907bc11cfad1777351e65
```

Send me the new result. If it still doesn't match, we'll investigate the line endings and whitespace rather than changing your code unnecessarily.

864b0a057e3cf9b1c5cdb7a3e1d12b18

Excellent! 🎉 **Test Case 3 MD5 is verified!**

Your Python program and the online calculator produced identical results.

Test Case 3	MD5
Python program	`864b0a057e3cf9b1c5cdb7a3e1d12b18`
Online calculator	`864b0a057e3cf9b1c5cdb7a3e1d12b18`
Status	✅ MATCH

This confirms that your program is hashing the multiline text correctly.

Next step: Verify SHA-1

Use the same lyrics in the online calculator, but select **SHA-1**.

Your Python program generated:

```text
a3b839313e70b8b5308c3402fe33773b79d35d33
```

Send me the online SHA-1 result, and we'll check it next!

a3b839313e70b8b5308c3402fe33773b79d35d33

Perfect! 🎉 **Test Case 3 SHA-1 is verified!**

Test Case 3	SHA-1
Python program	`a3b839313e70b8b5308c3402fe33773b79d35d33`
Online calculator	`a3b839313e70b8b5308c3402fe33773b79d35d33`
Status	✅ MATCH

So far, Test Case 3 has passed:

- MD5: ✅ MATCH
- SHA-1: ✅ MATCH

- SHA-256: ⏳ Still needs verification

Next step: Verify SHA-256

Use the same lyrics in your online calculator and select SHA-256.

Your Python program generated:

```
text
c0a69e523f131130b7d8ba1bcf8561599d8812cc7296a8abcd41569daea692b7
```

Send me the online calculator's SHA-256 result, and we'll finish Test Case 3!

c0a69e523f131130b7d8ba1bcf8561599d8812cc7296a8abcd41569daea692b7

Awesome, Mike! 🎉 **Test Case 3 is now completely verified!**

All three hashes generated by your Python program match the online calculator.

Test Case 3 — Verification Results

Algorithm	Python Program	Online Calculator	Result
MD5	`864b0a057e3cf9b1c5cdb7a3e1d12b18`	Matches	✅ PASS
SHA-1	`a3b839313e70b8b5308c3402fe33773b79d35d33`	Matches	✅ PASS
SHA-256	`c0a69e523f131130b7d8ba1bcf8561599d8812cc7296a8abcd41569daea692b7`	Matches	✅ PASS

Overall Progress

Test Case	MD5	SHA-1	SHA-256
Test Case 1	✅	✅	✅
Test Case 2	✅	✅	✅
Test Case 3	✅	✅	✅

Test Case	MD5	SHA-1	SHA-256
Test Case 4	⏳	⏳	⏳
Test Case 5	⏳	⏳	⏳

You've successfully verified 9 out of 9 hashes across the first three test cases!

Next: Test Case 4

The instructor's test document lists:

```
Pi = 3.14159
```

Enter that exact text into your Spyder program and generate the three hashes.

Send me the results, and we'll verify them one at a time.

One note for your final assignment: we've been using SHA-1 as a temporary interpretation of the instructor's "SHA-128" wording. SHA-1 is not SHA-128, so that terminology should be clarified before submission.

--- Hash Results ---
MD5: d77ef6f4f8992878c9950000b23948d4
SHA-1: 9e0d831752fbdd7e4d89264039b2032fe37de9a7
SHA-256:
2cbef46d6d3411a7dbb8087e6a25b01020bf6397ee7c93bfdc37e2e6
b0a9f20b

Great! You've generated all three hashes for **Test Case 4: `Pi = 3.14159`**.

Test Case 4 — Python Results

Algorithm	Generated Hash
MD5	`d77ef6f4f8992878c9950000b23948d4`
SHA-1	`9e0d831752fbdd7e4d89264039b2032fe37de9a7`

Algorithm	Generated Hash
SHA-256	`2cbef46d6d3411a7dbb8087e6a25b01020bf6397ee7c93bfdc37e2e6b0a9f20b`

Next: Verify MD5

Copy the exact text below into your online MD5 calculator:

```
Pi = 3.14159
```

Your expected MD5 result is:

```
d77ef6f4f8992878c9950000b23948d4
```

Send me the online calculator's result, and we'll verify it before moving to SHA-1.

d77ef6f4f8992878c9950000b23948d4

Perfect! **Test Case 4 MD5 is verified!** ✅

Your Python program and online calculator produced identical results.

Test Case 4	MD5
Python program	`d77ef6f4f8992878c9950000b23948d4`
Online calculator	`d77ef6f4f8992878c9950000b23948d4`
Status	✅ PASS

Next: Verify SHA-1

Use the same text:

```
```

```
Pi = 3.14159
```

Your Python program generated:

```
text
9e0d831752fbdd7e4d89264039b2032fe37de9a7
```

Send me the online calculator's SHA-1 result, and we'll check it!

9e0d831752fbdd7e4d89264039b2032fe37de9a7

Excellent! **Test Case 4 SHA-1 is verified!** ✅

Test Case 4	SHA-1
Python program	`9e0d831752fbdd7e4d89264039b2032fe37de9a7`
Online calculator	`9e0d831752fbdd7e4d89264039b2032fe37de9a7`
Status	✅ PASS

Next: Verify SHA-256

Use the same text:

```
text
Pi = 3.14159
```

Your Python program generated:

```
text
2cbef46d6d3411a7dbb8087e6a25b01020bf6397ee7c93bfdc37e2e6b0a9f20b
```

Send me the online calculator's SHA-256 result, and we'll finish Test Case 4!

2cbef46d6d3411a7dbb8087e6a25b01020bf6397ee7c93bfdc37e2e6b0a9f20b

Fantastic! 🎉 **Test Case 4 is completely verified!**

All three hashes match the online calculator.

Test Case 4 — Verification Results

Algorithm	Python vs. Online Calculator	Result
MD5	Exact match	✅ PASS
SHA-1	Exact match	✅ PASS
SHA-256	Exact match	✅ PASS

Overall Assignment Progress

Test Case	MD5	SHA-1	SHA-256
1. `2 + 2 = 5`	✅	✅	✅
2. `Good Morning! How are you!!`	✅	✅	✅
3. Multiline lyrics	✅	✅	✅
4. `Pi = 3.14159`	✅	✅	✅
5. `Dark Side of the Moon`	⏳	⏳	⏳

You've verified 12 out of 12 hashes so far!

Next: Test Case 5

Enter this exact text into your Spyder program:

```text
Dark Side of the Moon
```

Generate the hashes and send me the results.

We'll verify the final three hashes, then we can put together your test results for submission.

Remember: we still need to clarify the assignment's use of "SHA-128" because our current implementation uses SHA-1.

--- Hash Results ---
MD5: 811a0d2d1ddcc4fa8ad420ac0ea75dfe
SHA-1: 6c14dfe155850824e5392f41de64eb4a3c5306b0
SHA-256:
76c80fef9f1f673c3d79f1972914dd8b0ec43db0d4408eda9e76ad6589bca987

Great! You've generated all three hashes for **Test Case 5: `Dark Side of the Moon`**.

Test Case 5 — Python Results

Algorithm	Generated Hash
MD5	`811a0d2d1ddcc4fa8ad420ac0ea75dfe`
SHA-1	`6c14dfe155850824e5392f41de64eb4a3c5306b0`
SHA-256	`76c80fef9f1f673c3d79f1972914dd8b0ec43db0d4408eda9e76ad6589bca987`

First: Verify MD5

Enter this exact text into your online MD5 calculator:

```text
Dark Side of the Moon
```

Your expected MD5 result is:

```text
811a0d2d1ddcc4fa8ad420ac0ea75dfe
```

Send me the online calculator's result, and we'll verify the final test case one algorithm at a time!

811a0d2d1ddcc4fa8ad420ac0ea75dfe

Perfect! **Test Case 5 MD5 is verified!** ✅

Your Python program and online calculator produced identical results.

Test Case 5	MD5
Python program	`811a0d2d1ddcc4fa8ad420ac0ea75dfe`
Online calculator	`811a0d2d1ddcc4fa8ad420ac0ea75dfe`
Status	✅ PASS

Next: Verify SHA-1

Use the same text:

```text
Dark Side of the Moon
```

Your Python program generated:

```text
6c14dfe155850824e5392f41de64eb4a3c5306b0
```

Send me the online calculator's SHA-1 result, and then we'll finish with SHA-256!

6c14dfe155850824e5392f41de64eb4a3c5306b0

Excellent! **Test Case 5 SHA-1 is verified!** ✅

Test Case 5	SHA-1
Python program	`6c14dfe155850824e5392f41de64eb4a3c5306b0`

Test Case 5	SHA-1
Online calculator	`6c14dfe155850824e5392f41de64eb4a3c5306b0`
Status	✅ PASS

Final verification: SHA-256

Use the same text:

```
Dark Side of the Moon
```

Your Python program generated:

```
76c80fef9f1f673c3d79f1972914dd8b0ec43db0d4408eda9e76ad6589bca987
```

Send me the online calculator's SHA-256 result. Once it matches, all five test cases will be verified across all three algorithms!

76c80fef9f1f673c3d79f1972914dd8b0ec43db0d4408eda9e76ad6589bca987

Fantastic, Mike! 🎉 **Test Case 5 is completely verified!**

Your Python program and online calculator produced matching results for all three algorithms.

Test Case 5 — Verification Results

Algorithm	Result
MD5	✅ PASS
SHA-1	✅ PASS
SHA-256	✅ PASS

Final Assignment Test Results

You've now completed all five test cases!

Test Case	MD5	SHA-1	SHA-256
1. `2 + 2 = 5`	✅	✅	✅
2. `Good Morning! How are you!!`	✅	✅	✅
3. Multiline lyrics	✅	✅	✅
4. `Pi = 3.14159`	✅	✅	✅
5. `Dark Side of the Moon`	✅	✅	✅
Total	**5/5**	**5/5**	**5/5**

Overall Score: 15/15 — 100% Verified!

You have successfully:

- Built a text hash calculator.
- Implemented MD5, SHA-1, and SHA-256 hashing.
- Added multiline text support.
- Built a file integrity checker.
- Verified all five instructor test cases against an online hash calculator.
- Successfully checked the three provided files against their trusted SHA-256 hashes (3/3).

One important item before submission

Your assignment specifies **SHA-128**, but our program currently uses SHA-1. These are not the same algorithm. SHA-1 produces a 160-bit hash, not a 128-bit hash.

I recommend confirming the instructor's intended algorithm before submitting.

My suggestion for the next step: Let's put together a professional test-results document showing all five test cases, their hashes, and verification results so you have documentation to submit with your Python program.